T3000 API & Protocol

PID Loops · built Nov 13, 2026

PID Loops

Proportional-Integral-Derivative control loops for precise automatic control.

Overview

PID loops provide precise automatic control by continuously calculating an error value as the difference between a desired setpoint and a measured process variable. The controller attempts to minimize the error by adjusting the control output.

PID Basics

Control Equation

```text
Output = P×Error + I×∫Error×dt + D×(dError/dt)
```

Where:

- **P** (Proportional): Immediate response to current error
- **I** (Integral): Eliminates steady-state error over time
- **D** (Derivative): Dampens oscillations and overshoot

PID Components

Proportional (P):

- Responds to current error magnitude
- Higher P = faster response, more aggressive
- Too high causes oscillation

Integral (I):

- Eliminates offset error
- Accumulates error over time
- Too high causes overshoot and instability

Derivative (D):

- Predicts future error based on rate of change
- Reduces overshoot and settling time
- Sensitive to noise

PID Loop Configuration

Basic Settings

Process:

- **Input**: Measured variable (e.g., Zone Temperature)
- **Output**: Control signal (e.g., Valve Position)
- **Setpoint**: Desired target value
- **Units**: Engineering units

Tuning Parameters:

```
TEXT

Proportional (P): 2.0
Integral (I):     0.5
Derivative (D):   0.1
```

Operating Range:

- **High Limit**: Maximum output (100%)
- **Low Limit**: Minimum output (0%)
- **Setpoint Range**: Allowed SP range

Advanced Settings

Deadband:

```
TEXT

Deadband: 1.0°F
```

No control action if error < deadband

Rate Limiting:

```
TEXT

Max Change: 10% per minute
```

Prevents rapid output changes

Anti-Windup:

```
TEXT

Integral Limit: ±20%
```

Prevents integral term accumulation

Common PID Applications

Temperature Control

Heating Control:

```
TEXT

Input:     Zone_Temperature
Setpoint:  72°F
Output:    Hot_Water_Valve
P: 2.0, I: 0.5, D: 0.1
```

Cooling Control:

```
TEXT

Input:     Supply_Air_Temp
Setpoint:  55°F
Output:    Chilled_Water_Valve
P: 1.5, I: 0.3, D: 0.05
```

Pressure Control

Static Pressure:

```
Input:     Duct_Static_Press
Setpoint:  1.5 in.wc
Output:    Supply_Fan_VFD
P: 3.0, I: 1.0, D: 0.2
```

Flow Control

VAV Box Control:

```
Input:     Airflow_CFM
Setpoint:  500 CFM
Output:    Damper_Position
P: 2.5, I: 0.8, D: 0.15
```

PID Tuning

Manual Tuning

Ziegler-Nichols Method:

1. Set I and D to zero
2. Increase P until output oscillates
3. Note critical gain (Ku) and period (Tu)
4. Calculate:

```
P = 0.6 × Ku
I = 0.5 × Tu
D = 0.125 × Tu
```

Trial and Error:

1. Start with conservative values:

```
P = 1.0, I = 0.1, D = 0.01
```

2. Adjust P:

- Increase if too slow
- Decrease if oscillating

3. Add I to eliminate offset:
   - Start small (0.1)
   - Increase gradually
4. Add D if oscillating:
   - Start small (0.01)
   - Increase to dampen

Tuning Guidelines

Response Types:

- **Underdamped**: Overshoots, oscillates (reduce P or I)
- **Critically Damped**: Fast, no overshoot (ideal)
- **Overdamped**: Slow, sluggish (increase P)

Common Issues:

Problem	Solution
Slow response	Increase P
Oscillation	Decrease P, increase D
Steady-state error	Increase I
Overshoot	Decrease P or I, increase D
Noise amplification	Decrease D, add filter

Monitoring PID Performance

Real-Time Monitoring

View PID loop performance:

Loop Status:

```
Setpoint:     72.0°F
Input:        71.8°F
Error:        -0.2°F
Output:       45%
P Term:       -0.4%
I Term:       -0.1%
D Term:       0.0%
```

Performance Metrics:

- **Settling Time**: Time to reach setpoint
- **Overshoot**: Maximum deviation beyond setpoint
- **Steady-State Error**: Persistent offset
- **Oscillation Period**: Cycle time if oscillating

Trend Analysis

Plot PID variables over time:

- Setpoint (blue line)
- Input (green line)
- Output (orange line)
- Error (red line)

Look for:

- Quick response to setpoint changes
- Minimal overshoot
- No oscillation
- Zero steady-state error

PID Loop Table

Manage all PID loops:

#	Label	Input	Setpoint	Output	Status	Mode
1	Zone_Temp_Loop	71.8°F	72.0°F	45%	✅ Normal	Auto
2	SA_Temp_Loop	55.2°F	55.0°F	62%	✅ Normal	Auto
3	SP_Control	1.48"	1.50"	58%	✅ Normal	Auto

Features

- Real-time parameter updates
- Auto/Manual mode switching
- Setpoint adjustment
- Parameter tuning interface
- Performance charts

Best Practices

Loop Design

1. **Choose Appropriate Sensor**
    - Fast response time
    - Accurate measurement
    - Proper location
2. **Size Actuator Correctly**
    - Adequate authority (range)
    - Fast enough response
    - Linear characteristics
3. **Set Realistic Limits**
    - Prevent unsafe conditions
    - Allow adequate range
    - Consider seasonal changes

Tuning Strategy

1. **Start Conservative**
    - Begin with low gains
    - Increase gradually
    - Test incrementally
2. **One Parameter at a Time**
    - Adjust P first
    - Then I
    - Finally D

3. **Document Settings**
    - Record final parameters
    - Note tuning date
    - Keep tuning history

Maintenance

1. **Regular Review**
    - Check loop performance monthly
    - Retune if needed
    - Update for seasonal changes
2. **Calibration**
    - Calibrate sensors annually
    - Verify output devices
    - Check mechanical linkages
3. **Performance Analysis**
    - Review trend data
    - Calculate metrics
    - Optimize settings

Troubleshooting

Poor Performance

Symptoms: Slow, oscillating, or unstable

Solutions:

1. Review tuning parameters
2. Check sensor calibration
3. Verify output device operation
4. Analyze trend data
5. Retune loop

Hunting

Symptoms: Continuous oscillation

Solutions:

1. Reduce P gain
2. Increase deadband
3. Add rate limiting
4. Check for mechanical issues
5. Increase D term (carefully)

Offset Error

Symptoms: Persistent error, doesn't reach setpoint

Solutions:

1. Increase I gain
2. Check for mechanical binding
3. Verify adequate output range
4. Ensure sensor accuracy

Next Steps

- Programs - Control programs
- Outputs - Output control
- Schedules - Setpoint scheduling

PID Loops

PID Controller Implementation

Full PID Class

TYPESCRIPT

```typescript
interface PIDConfig {
  kp: number;
  ki: number;
  kd: number;
  setpoint: number;
  outputMin: number;
  outputMax: number;
  deadband?: number;
  sampleTime?: number;  // milliseconds
}

class PIDController {
  private integral = 0;
  private lastError = 0;
  private lastTime = Date.now();
  private lastOutput = 0;

  constructor(private config: PIDConfig) {}

  update(processValue: number): number {
    const now = Date.now();
    const dt = (now - this.lastTime) / 1000; // seconds

    // Skip if sample time not elapsed
    if (this.config.sampleTime &&
        (now - this.lastTime) < this.config.sampleTime) {
      return this.lastOutput;
    }

    const error = this.config.setpoint - processValue;

    // Deadband
    if (this.config.deadband && Math.abs(error) < this.config.deadband) {
      return this.lastOutput;
```

```
    }

    // P term
    const pTerm = this.config.kp * error;

    // I term with anti-windup
    this.integral += error * dt;
    const iTerm = this.config.ki * this.integral;

    // D term on measurement (avoids derivative kick)
    const dInput = (processValue - (this.config.setpoint - this.lastError)) / dt;
    const dTerm = -this.config.kd * dInput;

    // Calculate output
    let output = pTerm + iTerm + dTerm;

    // Clamp output and handle anti-windup
    if (output > this.config.outputMax) {
      output = this.config.outputMax;
      this.integral -= error * dt;  // Anti-windup
    } else if (output < this.config.outputMin) {
      output = this.config.outputMin;
      this.integral -= error * dt;  // Anti-windup
    }

    // Update state
    this.lastError = error;
    this.lastTime = now;
    this.lastOutput = output;

    return output;
  }

  setSetpoint(setpoint: number) {
    this.config.setpoint = setpoint;
  }

  reset() {
    this.integral = 0;
    this.lastError = 0;
    this.lastOutput = 0;
```

```
  }

  getState() {
    return {
      integral: this.integral,
      lastError: this.lastError,
      output: this.lastOutput
    };
  }
}
```

Auto-Tuning Algorithm

TYPESCRIPT

```typescript
class PIDAutoTuner {
  async tune(input: string, output: string): Promise<PIDConfig> {
    console.log('Starting auto-tune...');

    // 1. Relay feedback test
    const relayAmplitude = 10;  // % output
    const measurements: number[] = [];
    const timestamps: number[] = [];

    // Run relay test for ~10 cycles
    for (let i = 0; i < 100; i++) {
      const pv = await this.readInput(input);
      measurements.push(pv);
      timestamps.push(Date.now());

      // Toggle output based on error
      const error = 72 - pv;  // Assume 72°F setpoint
      const outValue = error > 0 ? 50 + relayAmplitude : 50 - relayAmplitude;
      await this.writeOutput(output, outValue);

      await this.sleep(1000);  // 1 second samples
    }

    // 2. Analyze oscillations
    const analysis = this.analyzeOscillations(measurements, timestamps);

    // 3. Calculate PID parameters using Ziegler-Nichols
    const ku = (4 * relayAmplitude) / (Math.PI * analysis.amplitude);
    const tu = analysis.period / 1000;  // Convert to seconds

    return {
      kp: 0.6 * ku,
      ki: 1.2 * ku / tu,
      kd: 0.075 * ku * tu,
      setpoint: 72,
      outputMin: 0,
      outputMax: 100
    };
```

```
  }

  private analyzeOscillations(data: number[], times: number[]) {
    // Find peaks and calculate period
    const peaks: number[] = [];
    const peakTimes: number[] = [];

    for (let i = 1; i < data.length - 1; i++) {
      if (data[i] > data[i-1] && data[i] > data[i+1]) {
        peaks.push(data[i]);
        peakTimes.push(times[i]);
      }
    }

    const avgPeriod = peakTimes.length > 1 ?
      (peakTimes[peakTimes.length - 1] - peakTimes[0]) / (peakTimes.length - 1) :
      0;

    const avgAmplitude = peaks.reduce((a, b) => a + b, 0) / peaks.length;

    return {
      period: avgPeriod,
      amplitude: avgAmplitude
    };
  }
}
```

Cascade Control

TYPESCRIPT

```typescript
class CascadeController {
  private primary: PIDController;
  private secondary: PIDController;

  constructor(
    primaryConfig: PIDConfig,
    secondaryConfig: PIDConfig
  ) {
    this.primary = new PIDController(primaryConfig);
    this.secondary = new PIDController(secondaryConfig);
  }

  update(primaryPV: number, secondaryPV: number): number {
    // Primary loop output becomes secondary loop setpoint
    const secondarySetpoint = this.primary.update(primaryPV);
    this.secondary.setSetpoint(secondarySetpoint);

    // Secondary loop controls the actual output
    return this.secondary.update(secondaryPV);
  }
}

// Example: Zone temp controls supply temp setpoint,
// supply temp controls valve position
const cascade = new CascadeController(
  { kp: 2.0, ki: 0.5, kd: 0.1, setpoint: 72, outputMin: 50, outputMax: 65 },
  { kp: 3.0, ki: 1.0, kd: 0.2, setpoint: 55, outputMin: 0, outputMax: 100 }
);
```

PID Monitoring

TYPESCRIPT

```typescript
class PIDMonitor {
  logState(controller: PIDController, pv: number) {
    const state = controller.getState();

    console.log({
      timestamp: new Date().toISOString(),
      processValue: pv,
      setpoint: controller.config.setpoint,
      error: state.lastError,
      integral: state.integral,
      output: state.output,
      pTerm: controller.config.kp * state.lastError,
      iTerm: controller.config.ki * state.integral
    });
  }
}
```

Next Steps

- Outputs
- REST API
- Performance Tuning
- Performance Tuning - Optimization